



```
In [3]: !mkdir supply_chain_project
!dir
```

A subdirectory or file supply_chain_project already exists.

Volume in drive C is OS

Volume Serial Number is 0EDA-0C76

Directory of C:\Users\suman

```
01-11-2025  21:58    <DIR>          .
29-10-2025  17:57    <DIR>          ..
01-11-2025  16:18    <DIR>          .anaconda
01-11-2025  16:27    <DIR>          .conda
01-11-2025  16:19        25 .condarc
01-11-2025  16:18    <DIR>          .continuum
01-11-2025  21:57    <DIR>          .ipynb_checkpoints
01-11-2025  16:29    <DIR>          .ipython
01-11-2025  16:27    <DIR>          .jupyter
01-11-2025  19:57    <DIR>          .matplotlib
29-10-2025  18:17    <DIR>          .vscode
01-11-2025  16:27    <DIR>          anaconda3
29-10-2025  17:39    <DIR>          Contacts
01-11-2025  21:43    <DIR>          Data Engineer
01-11-2025  21:48      413,867 Data Engineer project.ipynb
01-11-2025  13:56    8,725,691 Data Engineer.zip
01-11-2025  21:54    480,994 data_analyst_supply_chain_project.ipynb
01-11-2025  13:56    4,039,856 Data_analyst_supply_chain_project.zip
01-11-2025  21:57        617 data_science_supply_chain_project.ipynb
01-11-2025  13:56    8,688,298 data_science_supply_chain_project.zip
01-11-2025  16:27    <DIR>          Documents
01-11-2025  21:55    <DIR>          Downloads
29-10-2025  17:39    <DIR>          Favorites
01-11-2025  20:48    22,912 final_supply_chain_analysis.csv
01-11-2025  19:25    <DIR>          learn flu
29-10-2025  17:39    <DIR>          Links
29-10-2025  17:39    <DIR>          Music
01-11-2025  19:54    <DIR>          OneDrive
01-11-2025  21:24    <DIR>          processed
29-10-2025  17:39    <DIR>          Saved Games
29-10-2025  17:57    <DIR>          Searches
01-11-2025  20:28    21,053 supply_chain_data.csv
01-11-2025  21:58    <DIR>          supply_chain_project
01-11-2025  21:46    413,867 Untitled.ipynb
01-11-2025  21:56        72 Untitled1.ipynb
01-11-2025  13:27    <DIR>          Videos
                11 File(s)      22,807,252 bytes
                25 Dir(s)  430,334,189,568 bytes free
```

```
In [4]: import os, zipfile

# 1 Create folders
os.makedirs("supply_chain_project/data", exist_ok=True)

# 2 Extract the ZIP file
```

```

with zipfile.ZipFile("data_science_supply_chain_project.zip", "r") as z:
    z.extractall("supply_chain_project/data")

# 3◇ List extracted files
for root, dirs, files in os.walk("supply_chain_project/data"):
    for f in files:
        print(os.path.join(root, f))

```

supply_chain_project/data\Data Science.pdf
 supply_chain_project/data\data_science_supply_chain_dataset.zip

In [6]: **import** os

```

for root, dirs, files in os.walk("supply_chain_project"):
    for f in files:
        print(os.path.join(root, f))

```

supply_chain_project\data\Data Science.pdf
 supply_chain_project\data\data_science_supply_chain_dataset.zip

In [7]: **import** zipfile, os

```

# Path to your inner dataset zip
dataset_zip = "supply_chain_project/data/data_science_supply_chain_dataset.zip"

# Extract it into a subfolder
with zipfile.ZipFile(dataset_zip, "r") as z:
    z.extractall("supply_chain_project/data/dataset")

# Check what's inside
for root, dirs, files in os.walk("supply_chain_project/data/dataset"):
    for f in files:
        print(os.path.join(root, f))

```

supply_chain_project/data/dataset\ds_inventory_data.csv
 supply_chain_project/data/dataset\ds_products_master.csv
 supply_chain_project/data/dataset\ds_sales_data.csv
 supply_chain_project/data/dataset\ds_shipment_data.csv

In [8]: **import** pandas **as** pd

```

# Load all CSV files
sales = pd.read_csv("supply_chain_project/data/dataset/ds_sales_data.csv")
inventory = pd.read_csv("supply_chain_project/data/dataset/ds_inventory_data.csv")
products = pd.read_csv("supply_chain_project/data/dataset/ds_products_master.csv")
shipments = pd.read_csv("supply_chain_project/data/dataset/ds_shipment_data.csv")

# Quick preview
print("Sales:")
display(sales.head())

print("Inventory:")
display(inventory.head())

print("Products:")

```

```
display(products.head())

print("Shipments:")
display(shipments.head())
```

Sales:

	Order_ID	Order_Date	Product_ID	Customer_ID	Region	Quantity	Unit_Price
0	O000001	2024-05-04	P00410	C9571	West	8	1240.51
1	O000002	2023-01-02	P01917	C6636	East	2	186.97
2	O000003	2024-06-11	P04068	C1804	East	1	1500.74
3	O000004	2024-09-29	P02860	C2647	West	16	411.38
4	O000005	2024-06-17	P07073	C2702	North	18	1895.57

Inventory:

	Warehouse_ID	Product_ID	Region	Stock_Quantity	Reorder_Level	Last_Update
0	W0037	P06251	West	1817	119	2024-06-
1	W0016	P06538	South	171	285	2024-02-
2	W0018	P05614	East	189	94	2024-07-
3	W0005	P03871	South	357	366	2024-05-
4	W0009	P08801	North	1187	105	2024-05-

Products:

	Product_ID	Category	Subcategory	Price
0	P00001	Food	Groceries	86.80
1	P00002	Sports	Tennis	1758.97
2	P00003	Clothing	Jeans	636.72
3	P00004	Sports	Tennis	266.67
4	P00005	Sports	Gym	942.24

Shipments:

	Shipment_ID	Order_ID	Region	Shipment_Date	Delivery_Date	Status	Delay
0	S000001	O052979	East	2024-06-26	2024-06-30	Delayed	
1	S000002	O006590	North	2024-03-04	2024-03-05	Delivered	
2	S000003	O007224	West	2024-01-15	2024-01-20	Delivered	
3	S000004	O061909	East	2023-02-17	2023-02-24	Delayed	
4	S000005	O070287	East	2024-09-16	2024-09-21	Delivered	

In [9]: `for name, df in {`

```
    "Sales": sales,  
    "Inventory": inventory,  
    "Products": products,  
    "Shipments": shipments  
}.items():  
    print(f"\n{name} info:")  
    print(df.info())  
    print(df.isna().sum())  
    print("-" * 50)
```

Sales info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 100000 entries, 0 to 99999

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Order_ID	100000 non-null	object
1	Order_Date	100000 non-null	object
2	Product_ID	100000 non-null	object
3	Customer_ID	100000 non-null	object
4	Region	100000 non-null	object
5	Quantity	100000 non-null	int64
6	Unit_Price	100000 non-null	float64
7	Discount	100000 non-null	float64
8	Total_Sales	100000 non-null	float64

dtypes: float64(3), int64(1), object(5)

memory usage: 6.9+ MB

None

Order_ID 0

Order_Date 0

Product_ID 0

Customer_ID 0

Region 0

Quantity 0

Unit_Price 0

Discount 0

Total_Sales 0

dtype: int64

Inventory info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 100000 entries, 0 to 99999

Data columns (total 6 columns):

#	Column	Non-Null Count	Dtype
0	Warehouse_ID	100000 non-null	object
1	Product_ID	100000 non-null	object
2	Region	100000 non-null	object
3	Stock_Quantity	100000 non-null	int64
4	Reorder_Level	100000 non-null	int64
5	Last_Updated	100000 non-null	object

dtypes: int64(2), object(4)

memory usage: 4.6+ MB

None

Warehouse_ID 0

Product_ID 0

Region 0

Stock_Quantity 0

Reorder_Level 0

Last_Updated 0

dtype: int64

```

Products info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Product_ID      10000 non-null  object
1   Category        10000 non-null  object
2   Subcategory     10000 non-null  object
3   Price           10000 non-null  float64
dtypes: float64(1), object(3)
memory usage: 312.6+ KB
None
Product_ID      0
Category        0
Subcategory     0
Price           0
dtype: int64
-----

```

```

Shipments info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Shipment_ID     100000 non-null  object
1   Order_ID        100000 non-null  object
2   Region          100000 non-null  object
3   Shipment_Date   100000 non-null  object
4   Delivery_Date   100000 non-null  object
5   Status          100000 non-null  object
6   Delay_Days      100000 non-null  int64
dtypes: int64(1), object(6)
memory usage: 5.3+ MB
None
Shipment_ID     0
Order_ID        0
Region          0
Shipment_Date   0
Delivery_Date   0
Status          0
Delay_Days      0
dtype: int64
-----

```

```
In [11]: sales.columns
```

```
Out[11]: Index(['Order_ID', 'Order_Date', 'Product_ID', 'Customer_ID', 'Region',
               'Quantity', 'Unit_Price', 'Discount', 'Total_Sales'],
              dtype='object')
```

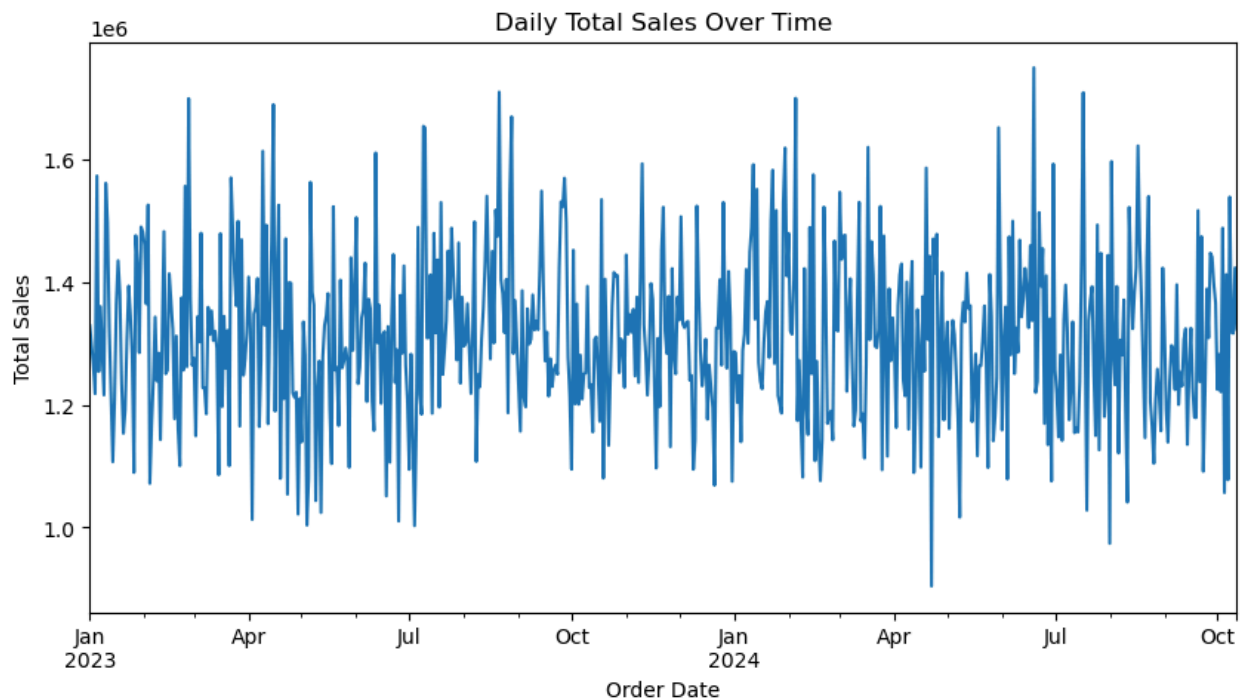
```
In [12]: import pandas as pd
import matplotlib.pyplot as plt
```

```

# Convert to datetime
sales['Order_Date'] = pd.to_datetime(sales['Order_Date'], errors='coerce')

# Plot daily total sales
sales.groupby('Order_Date')['Total_Sales'].sum().plot(
    figsize=(10,5),
    title="Daily Total Sales Over Time"
)
plt.ylabel("Total Sales")
plt.xlabel("Order Date")
plt.show()

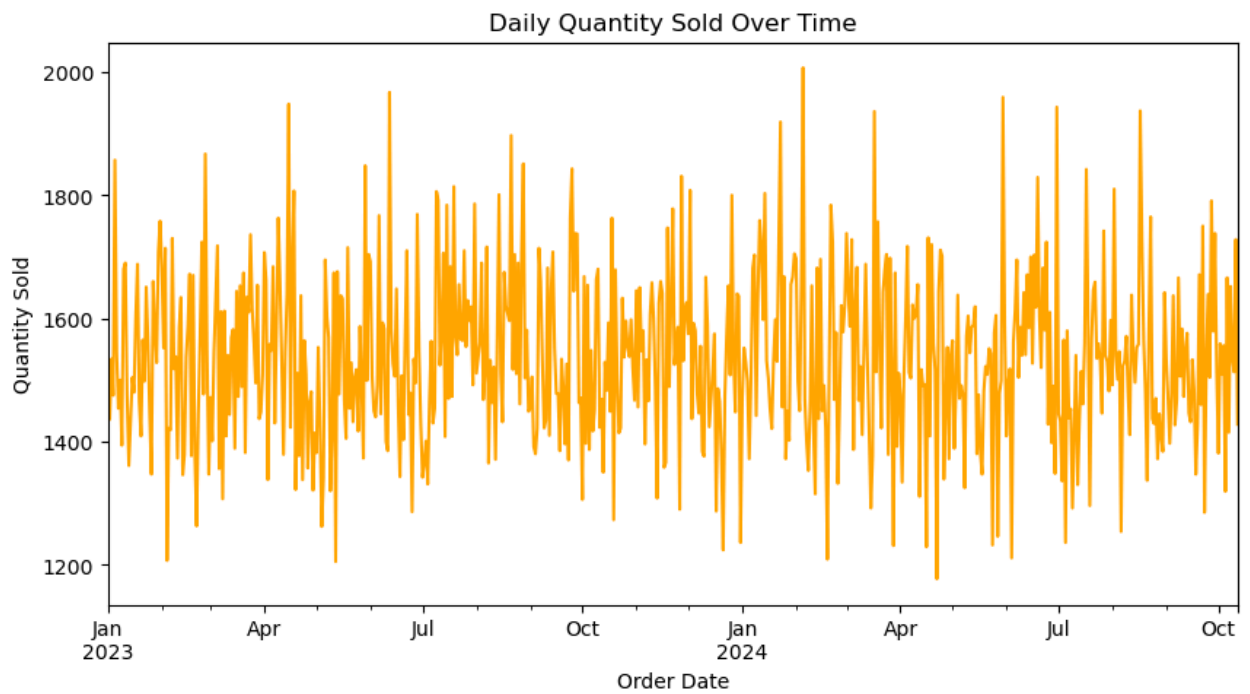
```



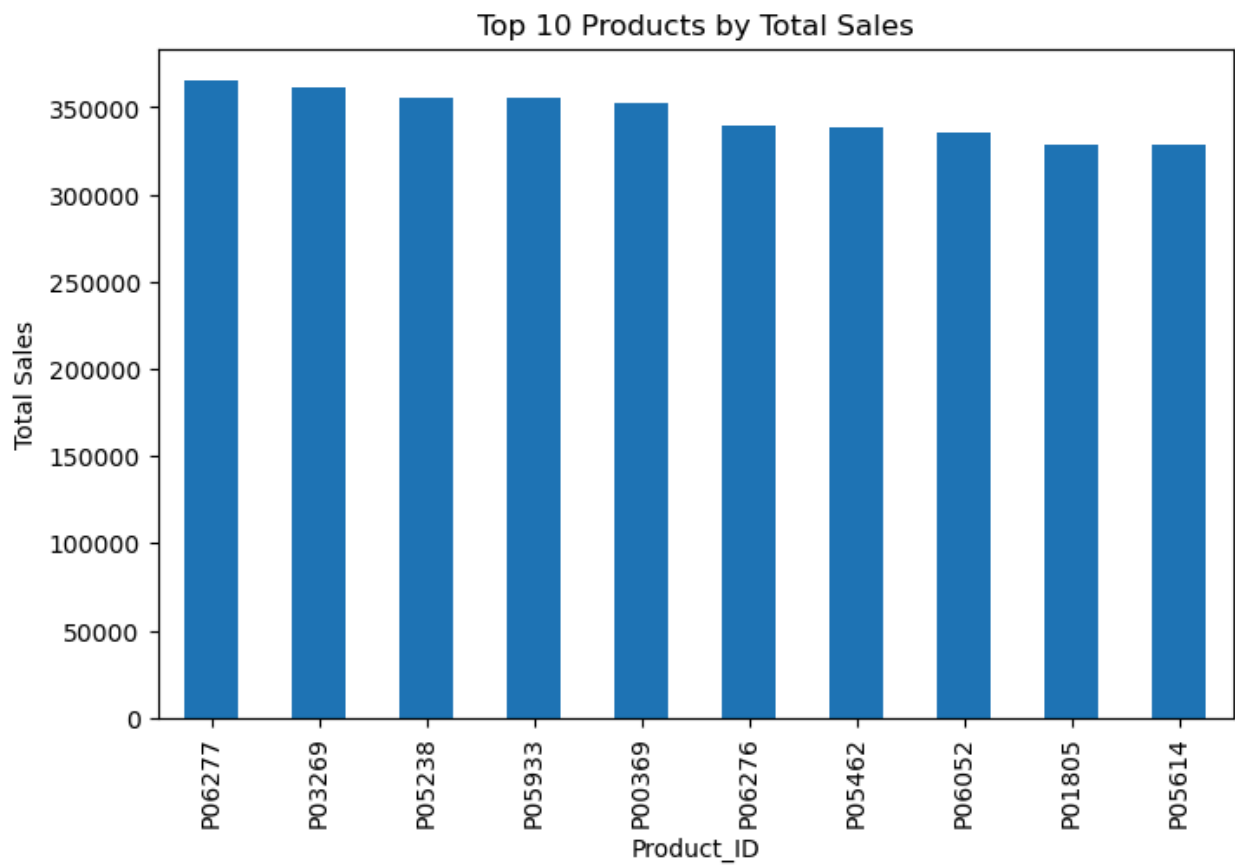
```

In [13]: sales.groupby('Order_Date')['Quantity'].sum().plot(
    figsize=(10,5),
    title="Daily Quantity Sold Over Time",
    color='orange'
)
plt.ylabel("Quantity Sold")
plt.xlabel("Order Date")
plt.show()

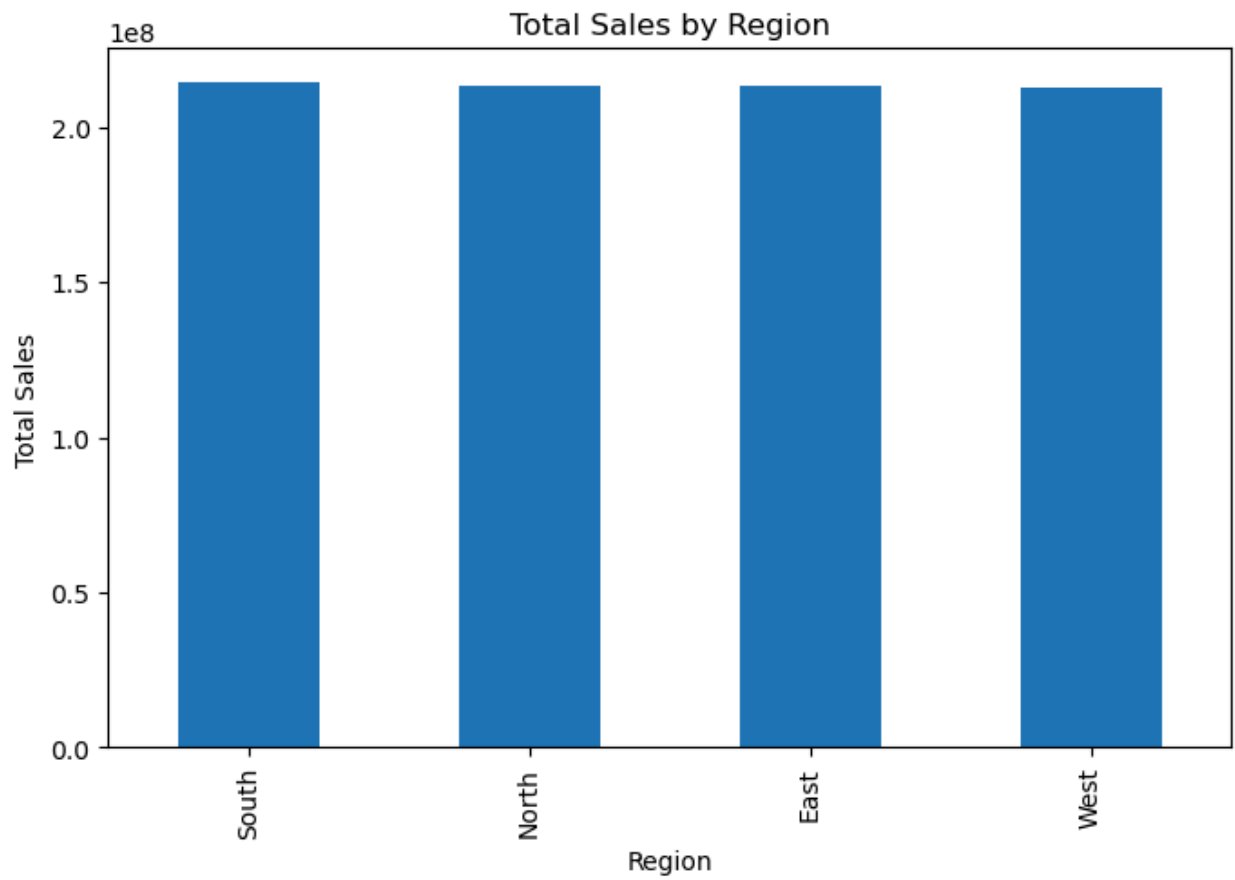
```



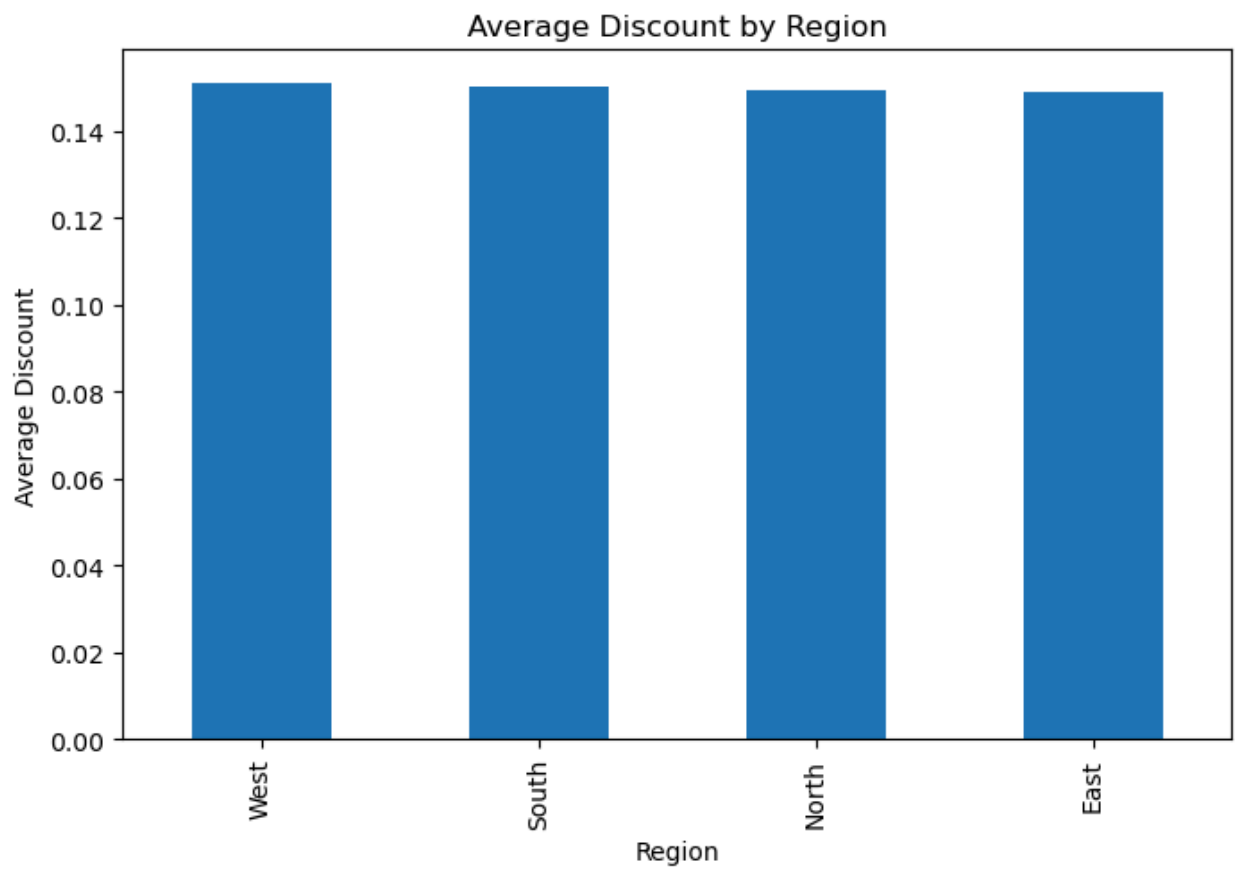
```
In [14]: top_products = (  
    sales.groupby('Product_ID')['Total_Sales']  
        .sum()  
        .sort_values(ascending=False)  
        .head(10)  
    )  
top_products.plot(kind='bar', title="Top 10 Products by Total Sales", figsize=  
plt.ylabel("Total Sales")  
plt.show()
```

```
In [15]: region_sales = sales.groupby('Region')['Total_Sales'].sum().sort_values(ascending=True)
region_sales.plot(kind='bar', title="Total Sales by Region", figsize=(8,5))
plt.ylabel("Total Sales")
plt.show()
```

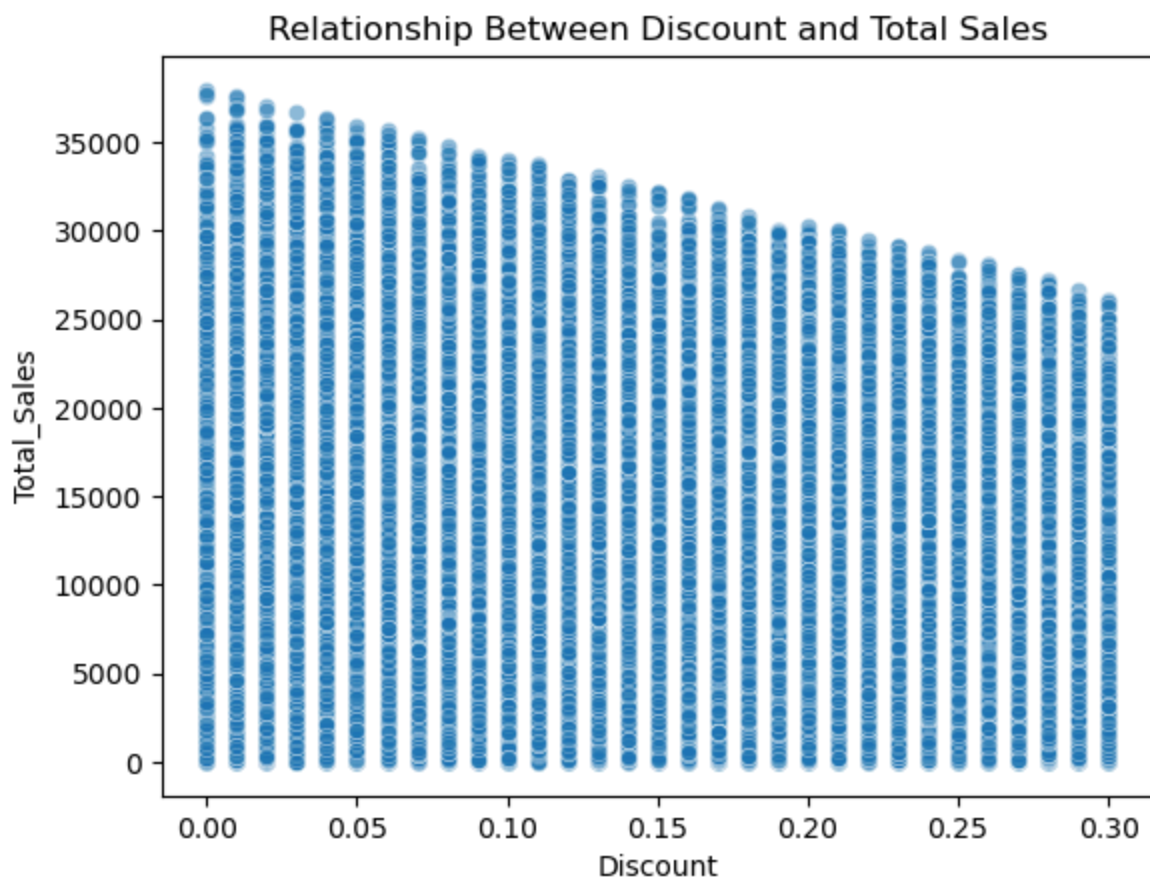


```
In [16]: discount_region = sales.groupby('Region')['Discount'].mean().sort_values(ascending=True)
discount_region.plot(kind='bar', title="Average Discount by Region", figsize=(10, 5))
plt.ylabel("Average Discount")
plt.show()
```

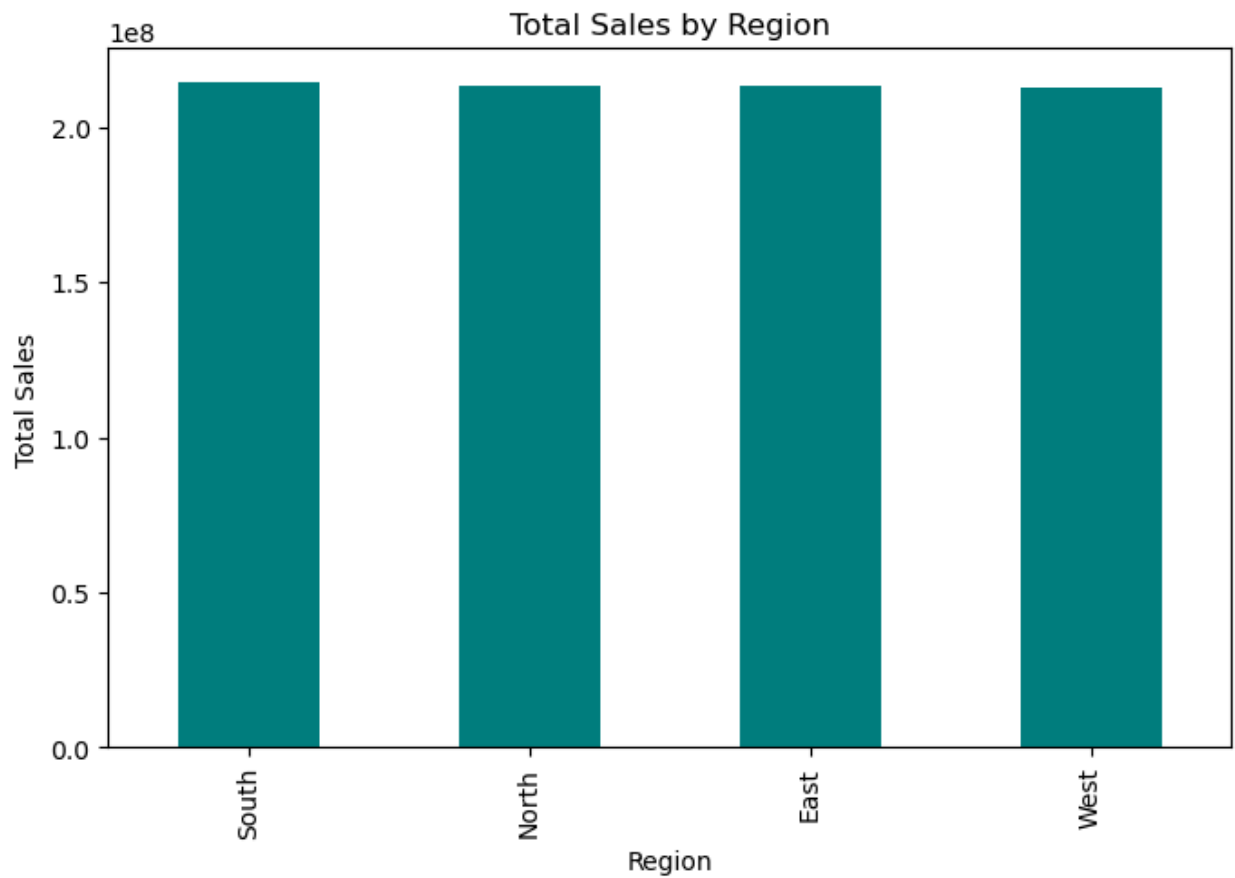


```
In [17]: import seaborn as sns
import matplotlib.pyplot as plt

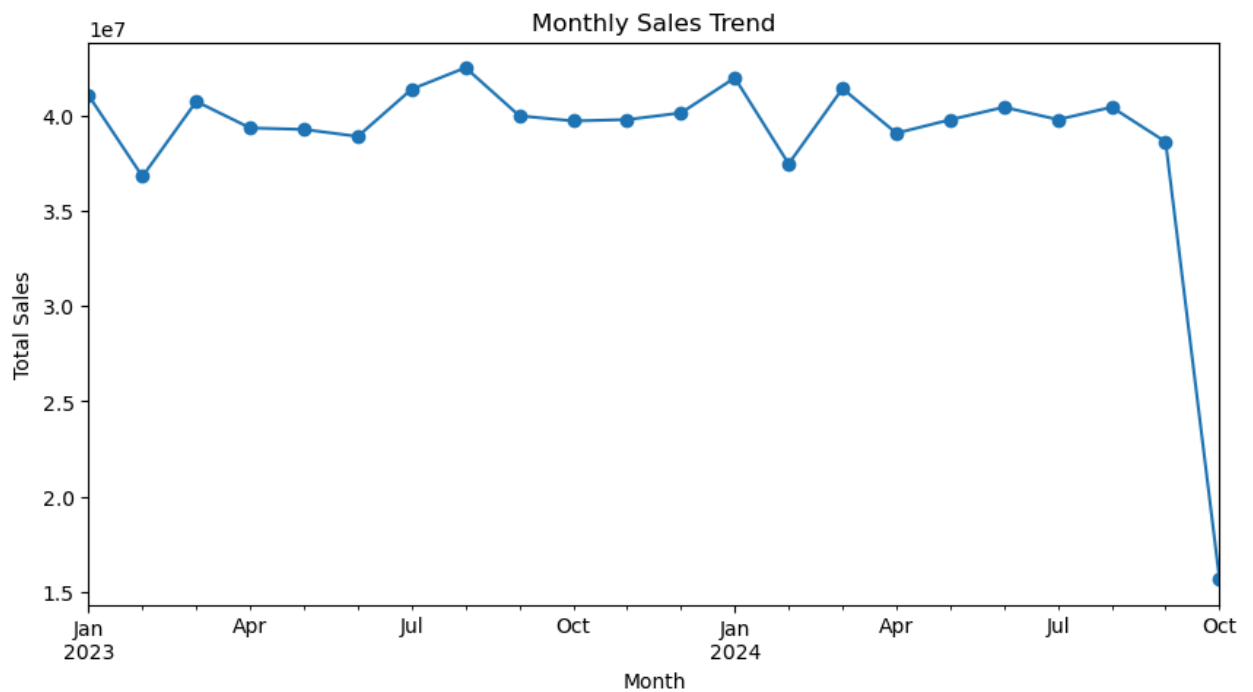
sns.scatterplot(x='Discount', y='Total_Sales', data=sales, alpha=0.5)
plt.title("Relationship Between Discount and Total Sales")
plt.show()
```



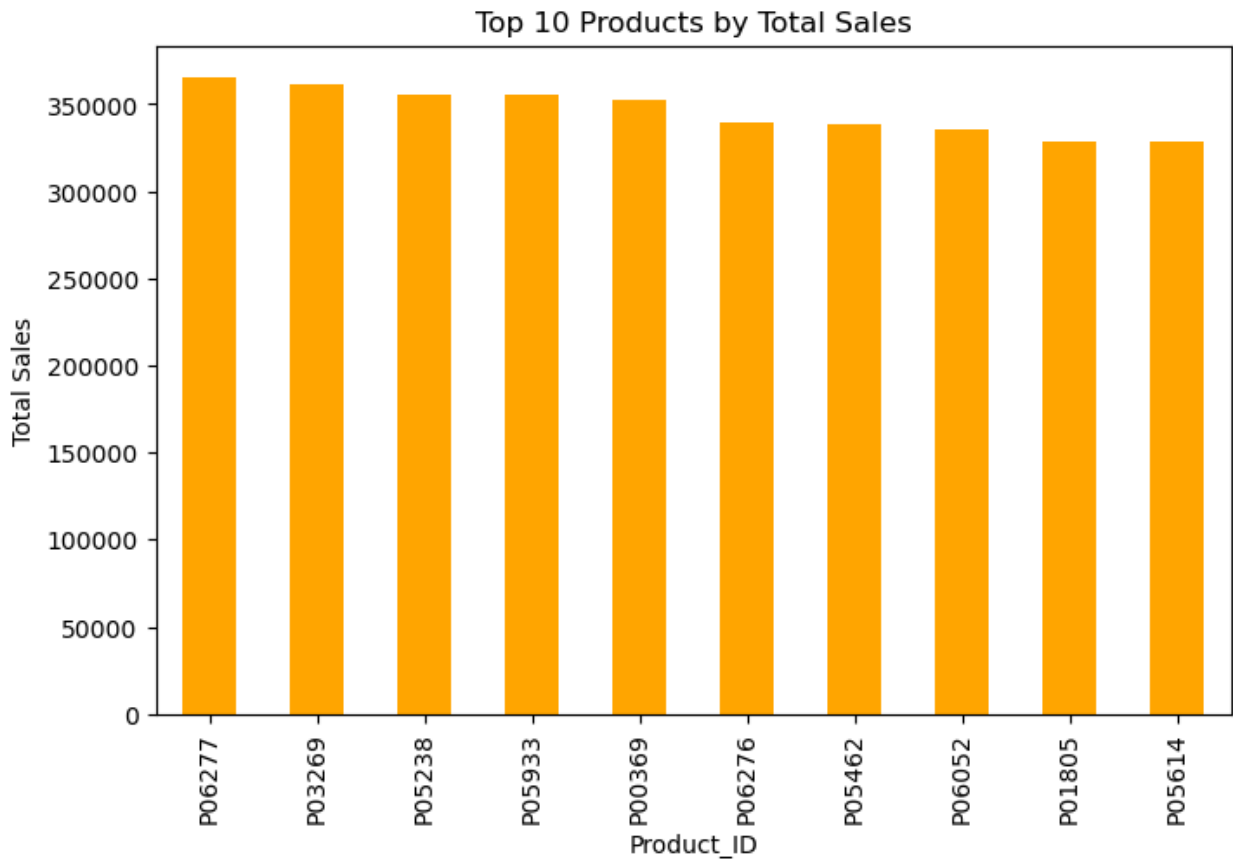
```
In [18]: region_sales = sales.groupby('Region')['Total_Sales'].sum().sort_values(ascending=True)
region_sales.plot(kind='bar', figsize=(8,5), title="Total Sales by Region", color='blue')
plt.ylabel("Total Sales")
plt.show()
```



```
In [19]: sales['Month'] = sales['Order_Date'].dt.to_period('M')
monthly_sales = sales.groupby('Month')['Total_Sales'].sum()
monthly_sales.plot(kind='line', figsize=(10,5), title="Monthly Sales Trend",
plt.ylabel("Total Sales")
plt.show()
```



```
In [20]: top_products = (
    sales.groupby('Product_ID')['Total_Sales']
        .sum()
        .sort_values(ascending=False)
        .head(10)
    )
top_products.plot(kind='bar', figsize=(8,5), title="Top 10 Products by Total S
plt.ylabel("Total Sales")
plt.show()
```



```
In [21]: daily_sales = sales.groupby('Order_Date')['Total_Sales'].sum().reset_index()
daily_sales.columns = ['ds', 'y'] # Prophet expects these names
daily_sales.head()
```

```
Out[21]:
```

	ds	y
0	2023-01-01	1328568.63
1	2023-01-02	1296383.40
2	2023-01-03	1263722.06
3	2023-01-04	1218004.73
4	2023-01-05	1572736.64

```
In [22]: pip install prophet
```

Collecting prophet

Downloading prophet-1.2.1-py3-none-win_amd64.whl.metadata (3.6 kB)

Collecting cmdstanpy>=1.0.4 (from prophet)

Downloading cmdstanpy-1.3.0-py3-none-any.whl.metadata (4.2 kB)

Requirement already satisfied: numpy>=1.15.4 in c:\users\suman\anaconda3\lib\site-packages (from prophet) (2.1.3)

Requirement already satisfied: matplotlib>=2.0.0 in c:\users\suman\anaconda3\lib\site-packages (from prophet) (3.10.0)

Requirement already satisfied: pandas>=1.0.4 in c:\users\suman\anaconda3\lib\site-packages (from prophet) (2.2.3)

Collecting holidays<1,>=0.25 (from prophet)

Downloading holidays-0.83-py3-none-any.whl.metadata (50 kB)

Requirement already satisfied: tqdm>=4.36.1 in c:\users\suman\anaconda3\lib\site-packages (from prophet) (4.67.1)

Collecting importlib_resources (from prophet)

Downloading importlib_resources-6.5.2-py3-none-any.whl.metadata (3.9 kB)

Requirement already satisfied: python-dateutil in c:\users\suman\anaconda3\lib\site-packages (from holidays<1,>=0.25->prophet) (2.9.0.post0)

Collecting stanio<2.0.0,>=0.4.0 (from cmdstanpy>=1.0.4->prophet)

Downloading stanio-0.5.1-py3-none-any.whl.metadata (1.6 kB)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (1.3.1)

Requirement already satisfied: cycler>=0.10 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (4.55.3)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (1.4.8)

Requirement already satisfied: packaging>=20.0 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (24.2)

Requirement already satisfied: pillow>=8 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (11.1.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\suman\anaconda3\lib\site-packages (from matplotlib>=2.0.0->prophet) (3.2.0)

Requirement already satisfied: pytz>=2020.1 in c:\users\suman\anaconda3\lib\site-packages (from pandas>=1.0.4->prophet) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in c:\users\suman\anaconda3\lib\site-packages (from pandas>=1.0.4->prophet) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\suman\anaconda3\lib\site-packages (from python-dateutil->holidays<1,>=0.25->prophet) (1.17.0)

Requirement already satisfied: colorama in c:\users\suman\anaconda3\lib\site-packages (from tqdm>=4.36.1->prophet) (0.4.6)

Downloading prophet-1.2.1-py3-none-win_amd64.whl (12.1 MB)

```
----- 0.0/12.1 MB ? eta -:--:--
-- ----- 0.8/12.1 MB 7.9 MB/s eta 0:00:02
----- 1.8/12.1 MB 4.7 MB/s eta 0:00:03
----- 2.9/12.1 MB 4.3 MB/s eta 0:00:03
----- 3.7/12.1 MB 4.2 MB/s eta 0:00:03
----- 4.5/12.1 MB 4.1 MB/s eta 0:00:02
----- 5.2/12.1 MB 4.1 MB/s eta 0:00:02
----- 6.0/12.1 MB 4.1 MB/s eta 0:00:02
----- 6.8/12.1 MB 4.0 MB/s eta 0:00:02
----- 7.9/12.1 MB 4.0 MB/s eta 0:00:02
----- 8.7/12.1 MB 4.0 MB/s eta 0:00:01
```


prophet-1.2.1 stanio-0.5.1

Note: you may need to restart the kernel to use updated packages.

In [24]: `from prophet import Prophet`

```
model = Prophet()
model.fit(daily_sales)

# Forecast 90 days into the future
future = model.make_future_dataframe(periods=90)
forecast = model.predict(future)
```

22:14:51 - cmdstanpy - INFO - Chain [1] start processing

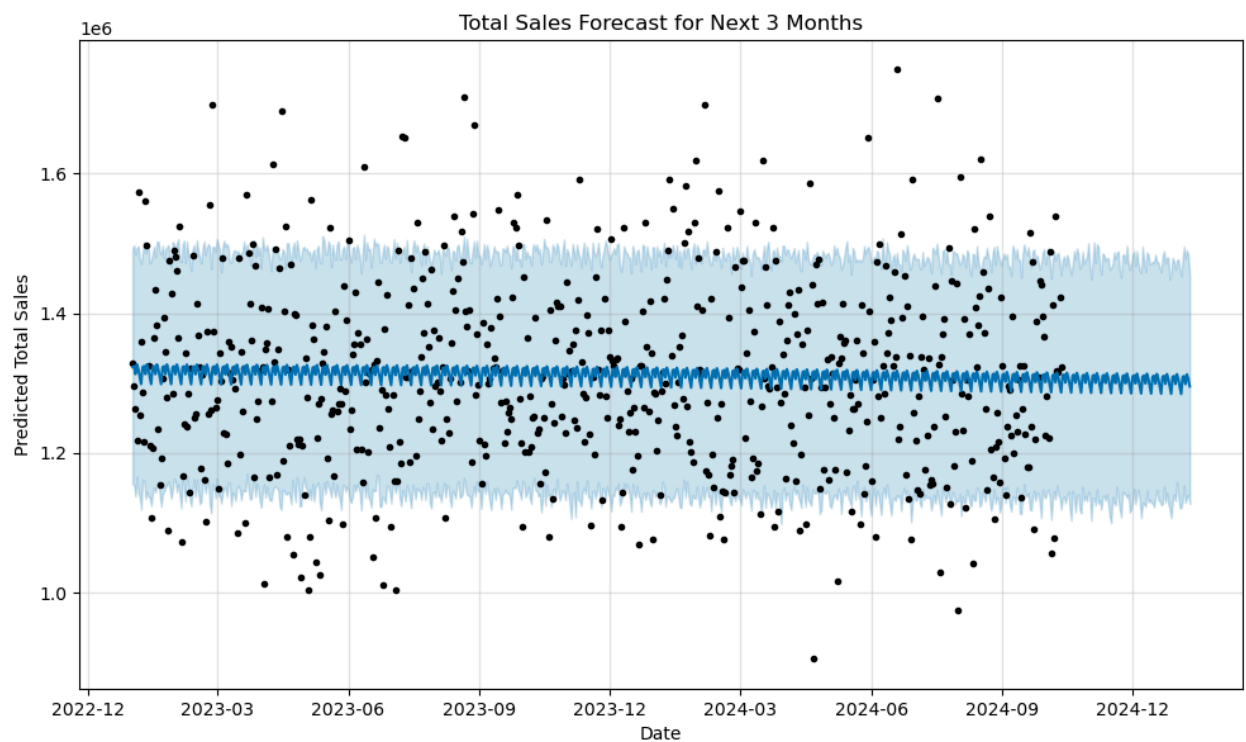
22:14:51 - cmdstanpy - INFO - Chain [1] done processing

In [25]: `future = model.make_future_dataframe(periods=90)`
`forecast = model.predict(future)`
`forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail()`

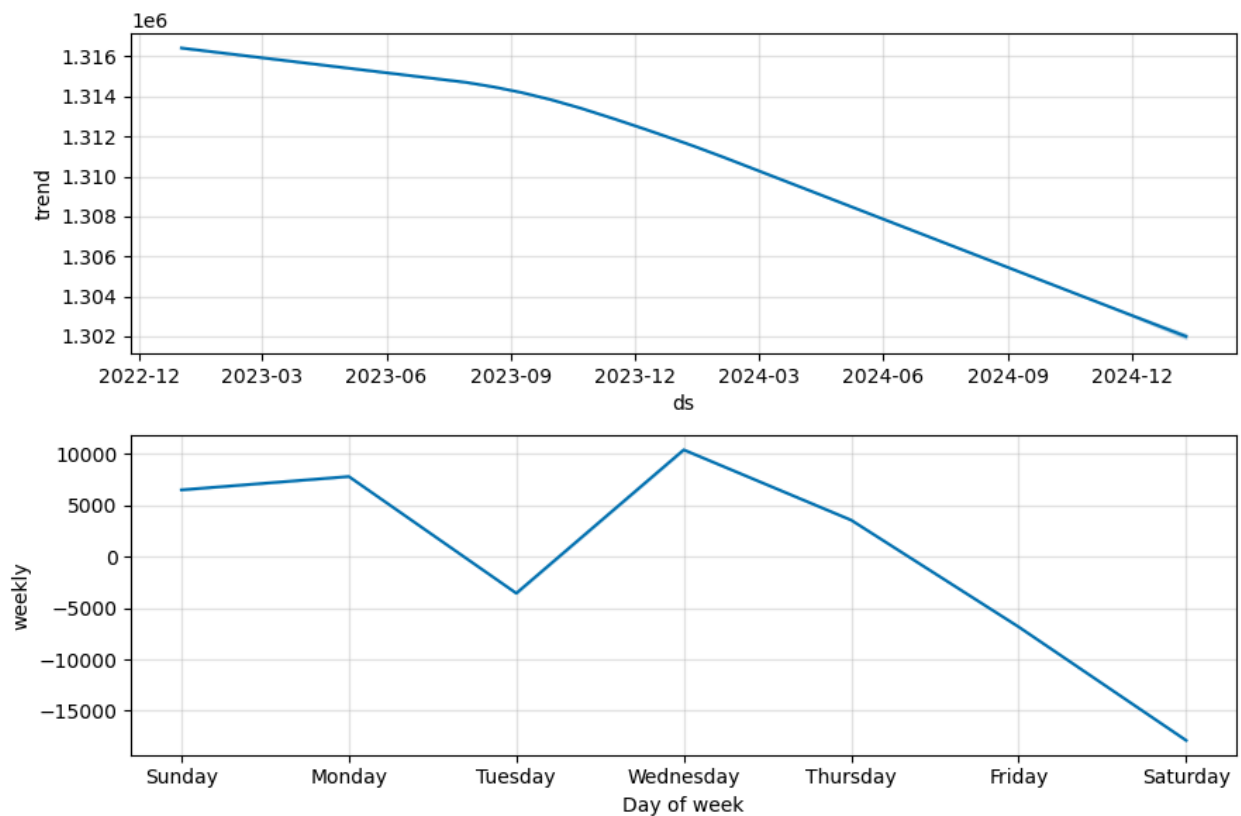
Out[25]:

	ds	yhat	yhat_lower	yhat_upper
736	2025-01-06	1.309908e+06	1.153271e+06	1.495384e+06
737	2025-01-07	1.298522e+06	1.128797e+06	1.477951e+06
738	2025-01-08	1.312455e+06	1.136768e+06	1.490384e+06
739	2025-01-09	1.305586e+06	1.140003e+06	1.483295e+06
740	2025-01-10	1.295170e+06	1.127123e+06	1.452927e+06

In [26]: `model.plot(forecast)`
`plt.title("Total Sales Forecast for Next 3 Months")`
`plt.xlabel("Date")`
`plt.ylabel("Predicted Total Sales")`
`plt.show()`



```
In [27]: model.plot_components(forecast)
plt.show()
```



```
In [28]: forecast.to_csv("supply_chain_project/data/sales_forecast.csv", index=False)
```

```
In [29]: import pandas as pd

forecast = pd.read_csv('supply_chain_project/data/sales_forecast.csv')
inventory = pd.read_csv('supply_chain_project/data/dataset/ds_inventory_data.csv')
products = pd.read_csv('supply_chain_project/data/dataset/ds_products_master.csv')

print(inventory.head())
```

	Warehouse_ID	Product_ID	Region	Stock_Quantity	Reorder_Level	Last_Updated
0	W0037	P06251	West	1817	119	2024-06-21
1	W0016	P06538	South	171	285	2024-02-27
2	W0018	P05614	East	189	94	2024-07-24
3	W0005	P03871	South	357	366	2024-05-27
4	W0009	P08801	North	1187	105	2024-05-28

```
In [30]: sales = pd.read_csv('supply_chain_project/data/dataset/ds_sales_data.csv')

daily_demand = (
    sales.groupby('Product_ID')['Quantity']
        .mean()
        .reset_index()
        .rename(columns={'Quantity': 'Average_Daily_Demand'})
)
```

```
In [31]: merged = pd.merge(inventory, daily_demand, on='Product_ID', how='left')
merged.head()
```

```
Out[31]:
```

	Warehouse_ID	Product_ID	Region	Stock_Quantity	Reorder_Level	Last_Updated	Average_Daily_Demand
0	W0037	P06251	West	1817	119	2024-06-21	1817
1	W0016	P06538	South	171	285	2024-02-27	171
2	W0018	P05614	East	189	94	2024-07-24	189
3	W0005	P03871	South	357	366	2024-05-27	357
4	W0009	P08801	North	1187	105	2024-05-28	1187

```
In [33]: inventory.columns
```

```
Out[33]: Index(['Warehouse_ID', 'Product_ID', 'Region', 'Stock_Quantity',
               'Reorder_Level', 'Last_Updated'],
              dtype='object')
```

```
In [34]: # You already have the merged dataset; if not, recreate it
merged = pd.merge(inventory, daily_demand, on='Product_ID', how='left')

# Determine if product needs reorder
merged['Need_Reorder'] = merged['Stock_Quantity'] < merged['Reorder_Level']

# Filter products that need reorder
to_reorder = merged[merged['Need_Reorder'] == True]
```

```
# Display only key columns
to_reorder[['Product_ID', 'Stock_Quantity', 'Reorder_Level', 'Average_Daily_De
```

Out[34]:

	Product_ID	Stock_Quantity	Reorder_Level	Average_Daily_Demand
1	P06538	171	285	9.363636
3	P03871	357	366	14.571429
36	P01666	252	274	11.090909
46	P05182	129	250	11.000000
54	P05439	180	195	6.000000
...	...	...	...	...
99953	P04006	158	217	9.454545
99955	P02179	74	75	9.000000
99979	P09742	50	63	6.333333
99981	P07179	78	443	9.461538
99995	P09699	339	349	10.909091

11537 rows × 4 columns

```
In [50]: # Load sales data
sales = pd.read_csv('supply_chain_project/data/dataset/ds_sales_data.csv')

# Convert date
sales['Order_Date'] = pd.to_datetime(sales['Order_Date'])

# Calculate daily demand per product
daily_demand = (
    sales.groupby('Product_ID')['Quantity']
    .mean()
    .reset_index()
    .rename(columns={'Quantity': 'Average_Daily_Demand'})
)

daily_demand.head()
```

Out[50]:

	Product_ID	Average_Daily_Demand
0	P00001	8.692308
1	P00002	9.545455
2	P00003	13.500000
3	P00004	10.142857
4	P00005	9.285714

```
In [51]: # Load inventory data
inventory = pd.read_csv('supply_chain_project/data/dataset/ds_inventory_data.csv')

# Merge both datasets
merged = pd.merge(inventory, daily_demand, on='Product_ID', how='left')

merged.head()
```

```
Out[51]:
```

	Warehouse_ID	Product_ID	Region	Stock_Quantity	Reorder_Level	Last_Update
0	W0037	P06251	West	1817	119	2024-01-01
1	W0016	P06538	South	171	285	2024-01-01
2	W0018	P05614	East	189	94	2024-01-01
3	W0005	P03871	South	357	366	2024-01-01
4	W0009	P08801	North	1187	105	2024-01-01

```
In [53]: to_reorder.loc[:, 'Priority_Score'] = to_reorder['Average_Daily_Demand'] / (to_reorder['Stock_Quantity'] - to_reorder['Reorder_Level'])
```

```
In [54]: to_reorder = merged[merged['Need_Reorder'] == True].copy()
to_reorder['Priority_Score'] = to_reorder['Average_Daily_Demand'] / (to_reorder['Stock_Quantity'] - to_reorder['Reorder_Level'])
```

```
In [55]: to_reorder = to_reorder.sort_values(by='Priority_Score', ascending=False)
to_reorder.head(10)
```

```
Out[55]:
```

	Warehouse_ID	Product_ID	Region	Stock_Quantity	Reorder_Level	Last_Update
92810	W0049	P01431	East	50	312	2024-01-01
92587	W0038	P03270	West	52	213	2024-01-01
13213	W0020	P04448	North	53	157	2024-01-01
99226	W0013	P05799	East	51	369	2024-01-01
81152	W0016	P00836	East	51	490	2024-01-01
85866	W0038	P02682	North	52	402	2024-01-01
63829	W0035	P07983	East	55	437	2024-01-01
43513	W0018	P08815	West	50	400	2024-01-01
20488	W0003	P00382	South	52	97	2024-01-01
7711	W0015	P00102	North	51	158	2024-01-01

```
In [56]: import matplotlib.pyplot as plt

top10 = to_reorder.head(10)
plt.figure(figsize=(10,5))
plt.bar(top10['Product_ID'], top10['Priority_Score'], color='red')
plt.title("Top 10 Products to Reorder Urgently")
```

```
plt.xlabel("Product ID")
plt.ylabel("Priority Score (Demand / Stock)")
plt.xticks(rotation=45)
plt.show()
```



```
In [57]: # Save main analysis outputs
merged.to_csv("supply_chain_project/data/final_merged_dataset.csv", index=False)
to_reorder.to_csv("supply_chain_project/data/final_reorder_priority_list.csv",
```

```
In [58]: total_products = len(merged)
need_reorder = len(to_reorder)
percent_reorder = round(need_reorder / total_products * 100, 2)

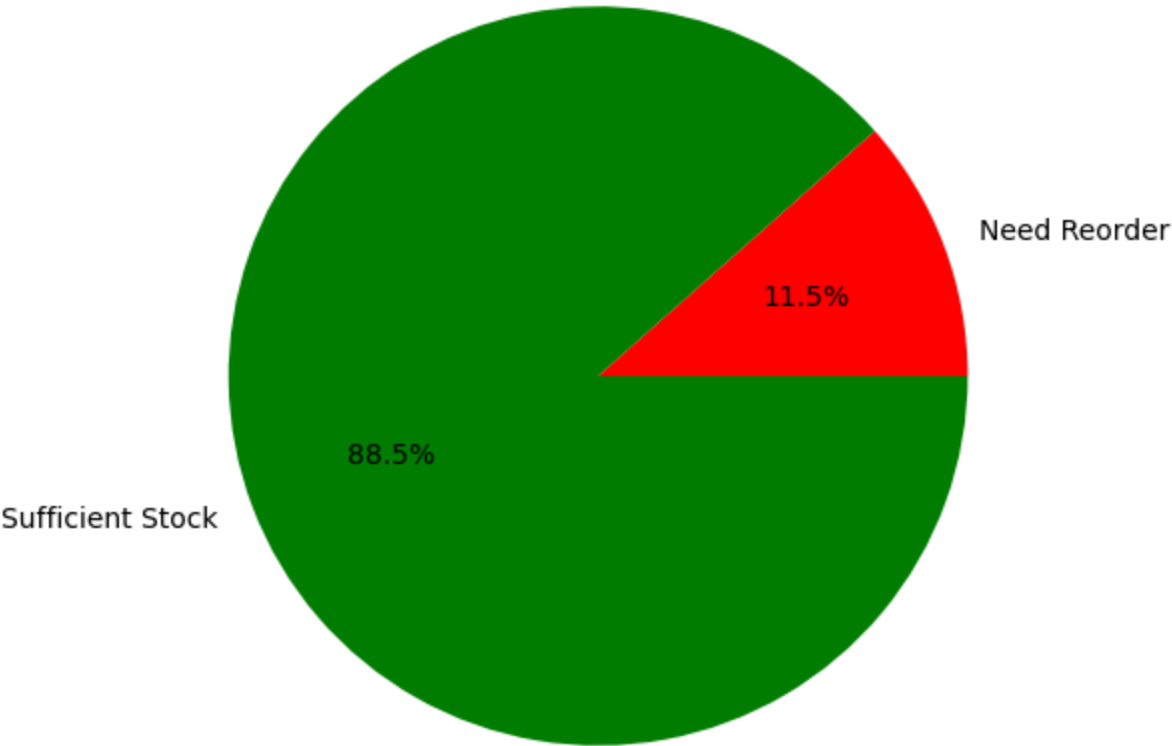
print(f"Total Products: {total_products}")
print(f"Products Needing Reorder: {need_reorder}")
print(f"Percentage Needing Reorder: {percent_reorder}%")
```

```
Total Products: 100000
Products Needing Reorder: 11537
Percentage Needing Reorder: 11.54%
```

```
In [59]: import matplotlib.pyplot as plt

plt.figure(figsize=(6,6))
plt.pie([need_reorder, total_products - need_reorder],
        labels=['Need Reorder', 'Sufficient Stock'],
        autopct='%1.1f%%',
        colors=['red', 'green'])
plt.title("Inventory Reorder Status Overview")
plt.show()
```

Inventory Reorder Status Overview



In []: